

Informe de Ingeniería Industrial: Patrón Invocador-Ejecutor para Aislamiento de Fallos en Entrenamiento YOLO Distribuido

William Steve Rodriguez Villamizar (wisrovi rodriguez) 
AI Leader & Solutions Architect
wisrovi-suit (<https://github.com/wisrovi/w-cli>)

1. Resumen y Palabras Clave

Resumen: Los procesos demonio persistentes que ejecutan PyTorch directamente en su propio espacio son vulnerables a fugas de memoria y kills OOM del kernel que causan inestabilidad del host. Este informe de experiencia industrial [1] documenta un estudio observacional de diseño del patrón Invocador-Ejecutor en `wyoloservice2`: un demonio Celery (Invocador) que nunca importa CUDA, y contenedores Docker efímeros (Ejecutores) con límites duros a nivel de SO (`mem_limit`, `nano_cpus`, `shm_size`). Comparamos cualitativamente este patrón contra ejecución directa, Ray, Kubernetes, containerd CRI, Kata, gVisor y Firecracker. El Invocador-Ejecutor contuvo exitosamente las fugas de memoria, registrando fallos vía eventos cgroups sin interrumpir el demonio. El patrón no es una invención novedosa pero su integración en una pila MLOps ligera produce una solución pragmática para estabilidad GPU.

Palabras Clave: Ingeniería Industrial, Aislamiento de Fallos, Aprendizaje Profundo Distribuido, Colas de Tareas Celery, Contenedores Efímeros, Container Runtimes.

2. Información del Autor

Este informe fue desarrollado por William Steve Rodriguez Villamizar (wisrovi rodriguez), AI Leader & Solutions Architect para wisrovi-suit (<https://github.com/wisrovi/w-cli>).

3. Introducción

Los clústeres de aprendizaje profundo sufren porque el demonio de entrenamiento es un punto único de fallo. Cuando un script YOLO filtra memoria, el OOM killer del kernel lo termina, dejando la GPU inconsistente y requiriendo reinicio. El patrón separa el plano de control (Invocador) del cómputo (Ejecutor efímero con límites duros). Al terminar, el contenedor se destruye (`docker run --rm`), liberando recursos.

4. Trabajo Relacionado y Líneas Base

Tiresias [2], Gandiva [3], AntMan [4] y Salus [5] optimizan recursos GPU, pero no fuerzan contenedorización efímera por tarea para prevenir caídas de demonios. Optimus [6] y Kubernetes [7] ofrecen gestión, pero con overhead. Ray [8] corre procesos persistentes. Las alternativas de runtimes de contenedores proporcionan garantías de aislamiento variables [9]. Firecracker [10] usa microVMs KVM para aislamiento fuerte. containerd [11] provee un runtime CRI. cgroups v2 [12] permite un control de grano fino. Kata Containers y gVisor [13] ofrecen aislamiento seguro a costa de latencia de arranque. NVIDIA GPU Operator [14] estandariza acceso.

5. Arquitectura Propuesta / Metodología

La arquitectura se representa en Figure 1. El demonio `wyoloservice2_invoker` corre en cada nodo. Al recibir tarea:

1. Deserializa payload YAML.
2. Calcula cuotas (`mem_limit`, `shm_size`).
3. Ejecuta

```
docker run --rm --gpus=all
--memory=${mem_limit} --cpus=${nano_cpus}
--shm-size=${shm_size}
wisrovi/train_service:worker_executor_v1.0.0
```
4. Captura código de salida y escribe en Redis.

Figura 1: Invocador genera contenedores Ejecutor efímeros por tarea.

6. Estudio Observacional de Diseño

Clúster: tres nodos RTX 4090, 64 GB RAM. Software: Celery 5.3, Docker 24.0, containerd 1.7, Kata Containers 3.0, gVisor, Firecracker 1.5, YOLOv8 [15]. Multiplexación vía MPS [16]. OOMs cualitativamente registrados con `dmesg/cgroups`.

7. Resultados y Discusión

7.1. Observaciones Cualitativas: Aislamiento Efímero

En nuestro estudio observacional, la ejecución directa causó inestabilidad del demonio host y requirió reinicios. Durante una ventana observacional de 14 días y aproximadamente 1,500 tareas de entrenamiento, Ray causó inestabilidad pero permitió que el driver de GPU se recuperara de forma autónoma en algunas ocasiones. Los entornos de contenedor aislaron los fallos cualitativamente, permitiendo que tareas individuales fallaran sin afectar aparentemente al demonio host. Kubernetes añadió una latencia de arranque notablemente mayor en nuestras observaciones; VMs también añadieron overhead de arranque medible en nuestra configuración. El patrón propuesto mantuvo la latencia baja ya que delega directamente al CLI de Docker. La verificación de diseño confirmó que el uso de memoria atípico típicamente resultó en la terminación vía `OOMKilled`, lo que generalmente evitó una cascada de inestabilidad en el proceso Invocador host.

8. Declaración de Disponibilidad de Datos y Código

Licencia Dual (PolyForm / AGPLv3). Los scripts y el código fuente residen en https://github.com/wisrovi/wyoloservice2_production.

9. Impacto Amplio / Declaración Ética

Prevenir caídas reduce el desgaste de hardware y mejora la eficiencia energética [17].

10. Conclusión y Trabajo Futuro

El patrón proporciona un aislamiento de fallos robusto para pipelines de entrenamiento YOLO. El tra-

bajo futuro explorará el perfilado de memoria en línea utilizando agentes LLM.

11. Agradecimientos

Gracias a los contribuyentes de wisrovi-suit.

Referencias

- [1] V. Garousi, M. Felderer, and M. V. M. Antyl^a, “The need for empirical evidence in software engineering,” *IEEE Software*, vol. 33, no. 1, pp. 68–75, 2016.
- [2] J. Gu *et al.*, “Tiresias: A gpu cluster manager for distributed deep learning,” *USENIX NSDI*, 2019.
- [3] W. Xiao *et al.*, “Gandiva: Introspective cluster scheduling for deep learning,” in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018.
- [4] —, “Antman: Dynamic scaling on GPU clusters for deep learning,” in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020.
- [5] P. Yu and M. Chowdhury, “Salus: Fine-grained GPU sharing primitives for deep learning applications,” in *Proceedings of the 3rd Conference on Machine Learning and Systems (MLSys)*, 2022.
- [6] Y. Peng *et al.*, “Optimus: an efficient dynamic resource scheduler for deep learning clusters,” in *Proceedings of the Thirteenth EuroSys Conference*, 2018, pp. 1–14.
- [7] B. Burns *et al.*, “Borg, omega, and kubernetes,” in *ACM Queue*, 2016.
- [8] P. Moritz *et al.*, “Ray: A distributed framework for emerging ai applications,” in *USENIX OSDI*, 2018.
- [9] T. Young *et al.*, “The true cost of containing: A performance study of container runtimes,” in *USENIX HotCloud*, 2019.
- [10] A. Agache *et al.*, “Firecracker: Lightweight virtualization for serverless applications,” *USENIX NSDI*, 2020.
- [11] M. Crosby *et al.*, “containerd: An industry-standard container runtime,” in *CNCF*, 2017.
- [12] T. Heo, “Control groups v2,” *Linux Kernel Documentation*, 2017.
- [13] Y. Wang *et al.*, “Performance and isolation analysis of runc, gvisor and kata containers,” *Cluster Computing*, 2022.
- [14] NVIDIA, “Nvidia gpu operator,” <https://github.com/NVIDIA/gpu-operator>, 2021.
- [15] G. Jocher *et al.*, “Ultralytics yolov8,” 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [16] NVIDIA, “Multi-process service (mps),” <https://docs.nvidia.com/deploy/mps/index.html>, 2023.
- [17] D. Patterson *et al.*, “Carbon emissions and large neural network training,” *arXiv preprint arXiv:2104.10350*, 2021.